

Ejercicio Evaluable 3: RPC

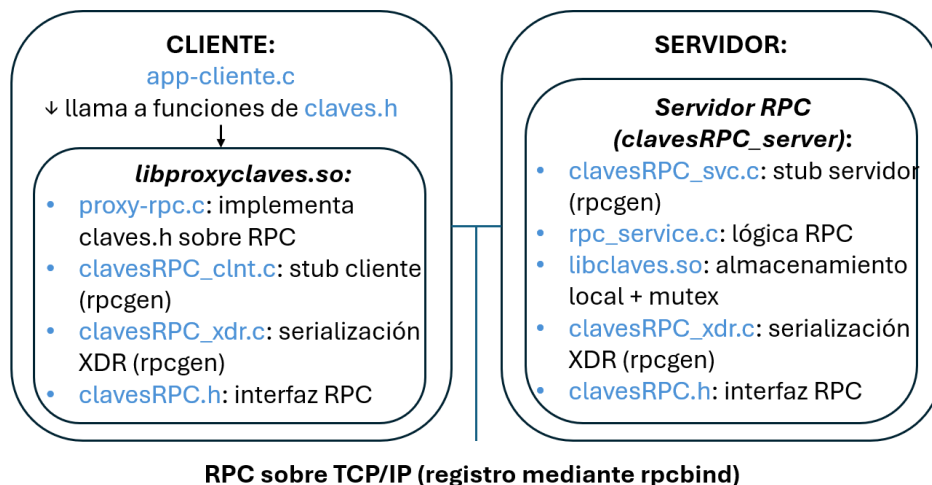
Sistemas Distribuidos 2025/26

Alicia Mei García Morín → 100498828
Ana Grima Vázquez de Prada → 100495785

1. Diseño realizado

1.1. Arquitectura general

La arquitectura sigue el modelo cliente-servidor clásico de RPC. El servidor aloja la lógica de negocio (almacenamiento de tuplas en memoria) y el cliente la consume a través de una biblioteca proxy transparente (libproxyclaves.so). La figura siguiente muestra el flujo de una llamada RPC completa:



1.2. Diseño de la interfaz RPC: `clavesRPC.x`

El fichero `clavesRPC.x` es el núcleo del diseño. Define los tipos XDR y los procedimientos remotos que rpcgen compilará. Se han definido tres estructuras:

- **PaqueteRPC:** equivalente a struct `Paquete` de `claves.h` (campos `x`, `y`, `z`).
- **ArgsSetModify:** agrupa los argumentos de `set_value` y `modify_value` (`key`, `value1`, `N_value2`, `V_value2`, `value3`).
- **GetValueResult:** respuesta de `get_value`, incluyendo un campo `status` que permite distinguir éxito de error sin perder los datos devueltos.

A partir del fichero `clavesRPC.x`, usamos `rpcgen` para generar automáticamente los ficheros necesarios para la comunicación remota (`clavesRPC.h`, `clavesRPC_clnt.c`, `clavesRPC_svc.c` y `clavesRPC_xdr.c`). Estos ficheros implementan los stubs cliente y servidor y la serialización de datos mediante XDR, permitiendo que las llamadas remotas se comporten como llamadas locales.

Durante las pruebas en el servidor compartido Guernika de la universidad, el identificador de programa RPC utilizado inicialmente (0x20000001), que usamos de ejemplos de rpcgen, ya estaba ocupado por otro servicio en el servidor. Por este motivo, se seleccionó un nuevo identificador único (0x20495785) para evitar conflictos en el registro del servicio.

1.3. Capa proxy cliente: proxy-rpc.c

El fichero proxy-rpc.c implementa las seis funciones declaradas en claves.h, satisfaciendo así el contrato original sin modificar ni un solo prototipo. La función auxiliar privada *crear_cliente_rpc()* centraliza la gestión del handle RPC y sigue el patrón siguiente para cada operación:

- Lee la variable de entorno IP_TUPLAS. Si no existe o está vacía, devuelve -1 inmediatamente.
- Crea un handle cliente RPC sobre TCP mediante *clnt_create()*.
- Empaqueta los argumentos en las estructuras XDR correspondientes.
- Invoca el stub generado por rpcgen (por ejemplo, *set_value_1()*).
- Comprueba el retorno RPC: si no es RPC_SUCCESS, cierra el handle y devuelve -1.
- Libera la memoria dinámica XDR con *xdr_free()* cuando es necesario (*get_value*). Este diseño garantiza que cualquier fallo de red o RPC se propaga como -1, cumpliendo la especificación que considera error también los fallos de comunicación

1.4. Capa de dispatching servidor: rpc_service.c

El fichero rpc_service.c contiene las funciones *_1_svc* que rpcgen espera. Cada una convierte los tipos RPC a los tipos locales (PaqueteRPC ↔ struct Paquete), delega en la función equivalente de libclaves.so y rellena la estructura de resultado.

1.5. Almacenamiento: lista enlazada dinámica con mutex POSIX

La lógica de datos (claves.c / libclaves.so) reutiliza íntegramente la implementación del ejercicio 1: una lista enlazada dinámica protegida por un mutex POSIX. Esta elección permite un número ilimitado de elementos en memoria y garantiza seguridad en acceso concurrente, ya que RPC lanza múltiples threads para atender peticiones simultáneas.

2. Estructura de archivos y compilación

2.1. Estructura de archivos

La siguiente tabla detalla los ficheros implementados y los ficheros generados automáticamente que intervienen en la compilación del proyecto.

Fichero	Origen	Rol
claves.h	Ejercicio 1 (sin cambios)	Define la API del servicio de claves
claves.c	Ejercicio 1 (sin cambios)	Implementa el almacenamiento local de claves
app-cliente.c	Ejercicio 1 (sin cambios)	Cliente de pruebas que invoca la API

clavesRPC.x	Nuevo	Define la interfaz RPC del servicio
clavesRPC.h	Generado por rpcgen	Cabecera con tipos, constantes y prototipos RPC
clavesRPC_clnt.c	Generado por rpcgen	Stub cliente RPC
clavesRPC_svc.c	Generado por rpcgen	Stub servidor/dispatcher RPC
clavesRPC_xdr.c	Generado por rpcgen	Serialización/deserialización XDR
proxy-rpc.c	Nuevo	Biblioteca cliente que adapta la API local a llamadas RPC
rpc_service.c	Nuevo	Implementación de las operaciones RPC del servidor
Makefile	Actualizado	Automatiza generación, compilación y enlazado del proyecto

2.2. Compilación de libclaves.so (servidor)

La biblioteca libclaves.so encapsula exclusivamente la lógica de almacenamiento local. Se compila como biblioteca dinámica (*-shared -fPIC*) y se enlaza con *-lpthread* para el soporte del mutex. Al ser una biblioteca dinámica, el servidor la carga en tiempo de ejecución, y el flag *-Wl,-rpath=.* en el Makefile indica al linker que la busque en el directorio de ejecución, evitando tener que instalarla en rutas del sistema.

2.3. Compilación de libproxyclaves.so (cliente)

La biblioteca libproxyclaves.so reúne tres objetos: proxy-rpc.o (la lógica proxy), clavesRPC_clnt.o (el stub cliente de rpcgen) y clavesRPC_xdr.o (la serialización XDR). Se enlaza con *-ltirpc*, que proporciona la implementación moderna de RPC en Linux (reemplazo de la antigua librpc de glibc). La opción de compilación *-I/usr/include/tirpc* asegura que las cabeceras RPC se encuentran correctamente.

2.4. Compilación del ejecutable del servidor

El ejecutable clavesRPC_server se construye enlazando los objetos del stub servidor (clavesRPC_svc.o), el módulo de dispatching (rpc_service.o) y la serialización XDR (clavesRPC_xdr.o), contra las bibliotecas libclaves.so (almacenamiento), tirpc (RPC) y pthread (mutex).

2.5. Compilación del ejecutable del cliente

El ejecutable cliente se genera a partir de app-cliente.o enlazado contra libproxyclaves.so y tirpc. La opción *-Wl,-rpath=.* asegura que la biblioteca proxy se resuelve en tiempo de ejecución desde el directorio actual.

2.6. Makefile

El Makefile organiza todo el proceso de compilación mediante reglas dependientes. La regla `all` desencadena la compilación completa del sistema. La regla `rpc` garantiza que los ficheros generados por `rpcgen` se crean o actualizan antes de compilar el resto de los componentes, a partir del fichero `clavesRPC.x`. Por último, la regla `clean` elimina los objetos, bibliotecas, ejecutables y ficheros generados automáticamente por `rpcgen`, que se regeneran al ejecutar `make`.

3. Ejecución del sistema

3.1. Arranque del servidor

En caso de que haya que eliminar los ficheros generados durante la compilación: `make clean`. Para compilar el proyecto completo, ejecutamos desde el directorio raíz del proyecto: `make`.

3.2. Ejecución de cliente y servidor

El servidor se ejecuta con: `./clavesRPC_server`.

Antes de lanzar el servidor es necesario que el servicio `rpcbind` (o `portmapper`) esté activo en el sistema. El servidor registra el programa `CLAVESRPC_PROG` (número `0x20495785`) en el `portmapper` local. A partir de ese momento queda bloqueado en el bucle de eventos de `rpcgen` esperando peticiones entrantes. No se requiere argumento alguno en línea de órdenes, a diferencia del servidor de sockets del ejercicio 2, donde era necesario especificar el puerto.

Por otro lado, el cliente debe conocer la dirección IP del servidor. Se utiliza la variable de entorno `IP_TUPLAS`, que `proxy-rpc.c` lee en cada llamada mediante `getenv()`. En otra terminal, hacemos: `export IP_TUPLAS=127.0.0.1` y `./cliente`.

El sistema permite ejecutar múltiples clientes de forma concurrente usando: `for i in {1..x}; do env IP_TUPLAS=127.0.0.1 ./cliente & done; wait`, donde `x` es el número de clientes que se desee.

4. Pruebas de funcionamiento

Por un lado, la batería de pruebas en `app-cliente.c` cubre los casos principales de la API: `exist` sobre clave inexistente (retorna 0), `get_value` sobre clave inexistente (retorna -1), `set_value`, `get_value` con verificación de todos los campos, `modify_value`, `delete_key` y comprobación final de inexistencia.

Por otro lado, se verificaron tres escenarios de fallo y en todos ellos el sistema respondió correctamente.

Escenario	Resultado
IP_TUPLAS no definida (unset IP_TUPLAS)	<pre>=== BATERIA DE PRUEBAS (RPC) === Error: la variable de entorno IP_TUPLAS no esta definida. [FAIL] exist("k1") inexistente got=-1 expected=0 El proxy devuelve -1.</pre>
Servidor apagado (IP válida, sin servicio)	<pre>=== BATERIA DE PRUEBAS (RPC) === 127.0.0.1: RPC: Remote system error - Connection refused [FAIL] exist("k1") inexistente got=-1 expected=0</pre>

	Error RPC Connection refused, se devuelve -1.
IP inexistente (192.168.1.99)	<pre>=== BATERIA DE PRUEBAS (RPC) === 192.168.1.99: RPC: Remote system error - Connection timed out [FAIL] exist("k1") inexistente got=-1 expected=0 Error RPC Connection timed out, se devuelve -1.</pre>

5. Trabajo extra

A lo largo de los ejercicios realizados se ha medido el tiempo total de ejecución del cliente bajo distintos niveles de concurrencia. Las pruebas se realizaron midiendo el tiempo total de ejecución de la batería de pruebas del cliente utilizando el comando `time bash -c 'for i in {1..x}; do env IP_TUPLAS=127.0.0.1 ./cliente & done; wait'` donde x es el número de clientes que se desea probar. Y se repitieron los experimentos con diferentes números de clientes concurrentes para analizar la escalabilidad del servidor. Obtuvimos los siguientes valores:

Número de clientes	EJ1 (Colas MQ)	EJ2 (Sockets TCP)	EJ3 (RPC)
1	0,005 s	0,014 s	0,025 s
5	0,006 s	0,019 s	0,055 s
20	0,0074 s	0,051 s	0,113 s
100	0,015 s	1,136 s	0,442 s

Llegamos a las siguientes conclusiones:

Con un único cliente, las colas de mensajes (ejercicio 1) presentan el menor tiempo de ejecución, ya que se ejecutan localmente y no requieren comunicación de red. Los sockets TCP (ejercicio 2) introducen un mayor coste debido al establecimiento de conexiones y al envío de datos a través de la red. RPC (ejercicio 3) añade una ligera sobrecarga adicional asociada a la serialización de datos y al uso de una capa de abstracción más compleja, lo que explica que su tiempo inicial sea mayor.

A medida que aumenta el número de clientes concurrentes, los tres sistemas muestran un incremento progresivo en el tiempo total de ejecución, lo que refleja el impacto de la concurrencia sobre el acceso a los recursos compartidos. Sin embargo, el crecimiento se mantiene controlado en todos los casos, lo que indica que las implementaciones son capaces de gestionar múltiples clientes de forma correcta.

El resultado más destacable se observa con un número elevado de clientes (100), donde la solución basada en RPC tiene un comportamiento más estable que la implementación con sockets TCP. Esto sugiere que, aunque RPC introduce una mayor complejidad inicial, ofrece un buen equilibrio entre facilidad de programación y capacidad para manejar múltiples clientes concurrentes.

En conjunto, los resultados muestran que cada tecnología presenta ventajas según el contexto: las colas de mensajes son muy eficientes en entornos locales, los sockets TCP ofrecen un control directo sobre la comunicación, y RPC proporciona una interfaz más sencilla y estructurada que facilita el desarrollo de aplicaciones distribuidas manteniendo un rendimiento adecuado.

*Nota:

- Se ha hecho uso de la IA (Github Copilot, ChatGPT) como herramienta de apoyo para la realización de la práctica, sin que en ningún momento ponga en duda nuestra autoría de esta (resolver dudas específicas, ayudar a la redacción de la memoria).